

High-Performance Paginated Data Iterator

Processing massive datasets from remote APIs or database cursors requires an efficient batching strategy to avoid memory exhaustion and minimize network overhead. A "Paginated Iterator" provides a seamless abstraction that allows developers to traverse large data volumes as a continuous stream while the underlying engine handles complex chunk-based retrieval.

Implement a `PaginatedIterator` class that wraps a remote data source using the Python iterator protocol. The engine must perform **lazy loading**: it should only fetch a new "page" of data when the current buffer is exhausted. The iterator must track the total volume of retrieved data and terminate gracefully when no more items are available.

Example 1 Input:

```
ITERATE 25 10
```

Output:

```
DEBUG: Fetching offset=0, limit=10, got 10 ... DEBUG: Fetching  
offset=10, limit=10, got 10 ... DEBUG: Fetching offset=20, limit=10,  
got 5 ... Total Fetched: 25
```

Explanation: The data source contains 25 items. With a page size of 10, the iterator performs three fetches (10, 10, and 5 items). The fourth fetch returns an empty list, signaling the end of the stream.

Example 2 Input:

```
ITERATE 5 10
```

Output:

```
DEBUG: Fetching offset=0, limit=10, got 5 Total Fetched: 5
```

Explanation: The entire dataset fits within a single page. The iterator fetches the first page, yields all 5 items, and then detects completion on the subsequent fetch.

Function Parameter:

- `data_source` (callable): A function that takes `(offset, limit)` and returns a list.
- `page_size` (int): The number of items to retrieve per batch.

Returns

- Any: Yields individual items from the data source sequentially.
- int: The cumulative `total_fetched` count available via the instance.

Constraints

- Implement both `__iter__` and `__next__` for full iterator protocol support.
- Fetching must be lazy (no data retrieved until first iteration).
- Correctly calculate the `offset` for each subsequent `data_source` call.

Input Format for Custom Testing Input from stdin contains: `ITERATE <total_items> <page_size>`.

Sample Case 0 Sample Input

```
ITERATE 12 5
```

Sample Output

```
DEBUG: Fetching offset=0, limit=5, got 5 DEBUG: Fetching offset=5,
limit=5, got 5 DEBUG: Fetching offset=10, limit=5, got 2 Total
Fetched: 12
```

Explanation The iterator fetches two full pages of 5 and one partial page of 2 to complete the 12-item traversal.